

Projek Struktur Data

Perancangan Sistem DSS Rekomendasi Game Menggunakan Struktur Data Graph berdasarkan Genre dan Rentang Harga



Tim Pengembang :

Bagaskara Refananda (2501010102)
Putu Mahesa Candradinata (2501010105)
I Putu Ryan Semara Putra (2501010106)

Institut Bisnis dan Teknologi Indonesia
Tahun Ajaran 2025-2026

Daftar Isi

Bab 1 Pendahuluan.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	1
1.3 Tujuan.....	1
1.4 Manfaat.....	2
1.4.1 Manfaat Teoretis.....	2
1.4.2 Manfaat Praktis.....	2
Bab 2 Dasar Teori.....	3
2.1 Struktur Data Graph.....	3
2.2 DSS.....	3
2.3 Algoritma Graph yang digunakan.....	3
2.3.1 Dijkstra.....	3
2.3.2 BFS (Breadth First-Search).....	4
2.3.3 SAW (Simple Additive Weighting).....	4
Bab 3 Analisis dan Perancangan.....	6
3.1 Analisis Masalah.....	6
3.2 Desain Graph.....	6
3.3 Flowchart.....	7
3.4 Use Case.....	8
3.4.1 Aktor yang Digunakan.....	8
3.4.2 Gambar Use Case.....	8
3.5 Struktur Node dan Edge.....	8
Bab 4 Implementasi.....	10
4.1 Implementasi Program.....	10
4.2 Penjelasan Kode.....	10
4.2.1 Struktur Data Game.....	10
4.2.2 Implementasi Graph.....	11
4.2.3 Implementasi BFS.....	11
4.2.4. Implementasi Dijkstra.....	11
4.2.5 Implementasi SAW.....	12
4.2.6. Implementasi GUI.....	12
4.3 Tampilan Sistem.....	13
4.3.1 Halaman Utama.....	13
4.3.2 Halaman Rekomendasi Game.....	13
4.3.3 Tampilan Output Filter.....	14
4.3.4 Pop Up Chart Visualisasi Graph.....	14
Bab 5 Pengujian dan Analisis.....	15
5.1 Skenario Pengujian.....	15
5.1.1 Semua genre, tanpa Filter Harga.....	15
5.1.2 Filter genre: Action.....	16
5.1.3 Filter Genre: RPG, harga gratis (min = 0, max = 0).....	16
5.1.4 Filter Genre: Simulation, Harga 0–200.000.....	17

5.1.5 Filter Genre: Fighting, Harga Maks 400.000.....	17
5.1.6 Multi-Genre: Sandbox + Shooter.....	18
5.1.7 Filter Menghasilkan Kandidat Kosong.....	18
5.2 Analisis Hasil.....	19
5.2.1 Alur Pipeline Rekomendasi.....	19
5.2.2 Hasil Skenario Tanpa Filter.....	19
5.2.3 Analisis Pengaruh Bobot SAW.....	20
5.2.4 Analisis Validitas Filter.....	21
5.2.5 Pengujian Antarmuka dan Responsivitas.....	21
5.3 Kompleksitas Program.....	22
5.3.1 Kompleksitas Keseluruhan Pipeline.....	22
5.3.2 Skalabilitas Sistem.....	23
Bab 6 Penutup.....	24
6.1 Kesimpulan.....	24
6.2 Saran Pengembang.....	24
DAFTAR PUSTAKA.....	25

Bab 1 Pendahuluan

1.1 Latar Belakang

Seiring dengan perkembangan zaman dan kemajuan teknologi yang semakin pesat, berbagai perubahan signifikan turut terjadi di berbagai sektor kehidupan, khususnya dalam dunia akademik dan sosial kemasyarakatan. Berdasarkan observasi awal yang dilakukan, permasalahan ini dinilai cukup krusial karena berdampak langsung terhadap efektivitas pembelajaran dan pembentukan karakter mahasiswa.

Ketertarikan penulis terhadap topik ini berawal dari keprihatinan melihat realita di lapangan yang tidak sesuai dengan harapan ideal. Penulis menyadari bahwa sebagai mahasiswa yang sedang menempuh mata kuliah ini, sudah selayaknya peka terhadap isu-isu aktual dan mampu mengkajinya secara ilmiah. Oleh karena itu, penulis memilih topik ini sebagai bahan kajian dalam laporan UAS, dengan harapan dapat memberikan kontribusi pemikiran yang konstruktif bagi perbaikan bersama, baik di lingkungan kampus maupun masyarakat luas.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan di atas, maka permasalahan yang akan dibahas dalam laporan ini dapat dirumuskan sebagai berikut:

1. Bagaimana merancang sistem pendukung keputusan (DSS) yang dapat memberikan rekomendasi game berdasarkan preferensi genre dan harga?
2. Metode atau algoritma apa yang paling sesuai untuk digunakan dalam sistem rekomendasi game dengan kriteria genre dan harga?
3. Seberapa akurat rekomendasi yang dihasilkan oleh sistem dibandingkan dengan pertimbangan manual dari pengguna?

1.3 Tujuan

Adapun tujuan dari penyusunan laporan ini adalah sebagai berikut:

1. Merancang dan membangun sistem DSS sederhana yang mampu memberikan rekomendasi game berdasarkan input genre dan rentang harga yang diinginkan pengguna.
2. Mengimplementasikan metode tertentu (seperti Simple Additive Weighting / SAW, Weighted Product, atau rule-based system) dalam proses pengambilan keputusan.

3. Menguji akurasi dan tingkat relevansi rekomendasi yang dihasilkan sistem terhadap kebutuhan nyata pengguna di kalangan mahasiswa.

1.4 Manfaat

Laporan ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis sebagai berikut:

1.4.1 Manfaat Teoretis

- Menambah referensi ilmiah di bidang sistem pendukung keputusan, khususnya yang diterapkan pada industri hiburan digital.
- Menjadi bahan kajian lebih lanjut bagi mahasiswa lain yang ingin mengembangkan DSS dengan kriteria yang lebih kompleks (misalnya: ulasan pengguna, rating usia, atau performa perangkat).

1.4.2 Manfaat Praktis

- Bagi penulis: Meningkatkan pemahaman dan keterampilan dalam mengimplementasikan konsep DSS ke dalam bentuk aplikasi sederhana yang fungsional.
- Bagi mahasiswa pada umumnya: Memberikan kemudahan dalam memilih game yang sesuai dengan minat dan anggaran, sehingga pembelian menjadi lebih terencana dan tidak boros.
- Bagi pengembang sistem di masa depan: Hasil laporan ini dapat menjadi prototipe awal yang kemudian dikembangkan menjadi aplikasi berbasis web atau mobile yang lebih matang dan komersial.

Bab 2 Dasar Teori

2.1 Struktur Data Graph

Terdapat dua jenis graph yang diimplementasikan, yaitu Graph Genre dan Graph Kemiripan Game.

Graph Genre merepresentasikan hubungan antar genre sebagai node, dengan edge yang terbentuk apabila dua genre pernah muncul bersama dalam satu kelompok game yang saling terhubung berdasarkan data EDGES_LAPORAN. Graph ini digunakan pada tahap BFS Traversal untuk menemukan kandidat game.

Graph Kemiripan merepresentasikan hubungan langsung antar game sebagai node, dengan edge berbobot yang menyatakan tingkat ketidakmiripan antar dua game. Graph ini digunakan pada tahap Dijkstra untuk menghitung jarak kemiripan minimum dari game acuan ke seluruh kandidat.

2.2 DSS

Decision Support System merupakan sistem yang membantu pengguna dalam mengambil keputusan berdasarkan data dan kriteria tertentu. DSS tidak menggantikan pengambilan keputusan manusia, melainkan memberikan alternatif yang dapat dipertimbangkan.

2.3 Algoritma Graph yang digunakan

2.3.1 Dijkstra

Algoritma Dijkstra merupakan algoritma pencarian jalur terpendek (shortest path) pada sebuah graph berbobot yang diperkenalkan oleh Edsger W. Dijkstra pada tahun 1956. Algoritma ini bekerja dengan mencari jarak minimum dari satu node sumber menuju node-node lainnya berdasarkan bobot yang dimiliki setiap sisi (edge) pada graph.

Dalam DSS rekomendasi game, algoritma Dijkstra digunakan untuk mencari game yang paling relevan dengan preferensi pengguna. Setiap game direpresentasikan sebagai node dan hubungan antar game sebagai edge berbobot berdasarkan kesamaan genre, harga, rating, atau atribut lainnya. Game dengan bobot atau jarak terkecil dianggap memiliki tingkat kemiripan tertinggi sehingga lebih direkomendasikan kepada pengguna.

Langkah Kerja Algoritma Dijkstra

1. Menentukan node awal berdasarkan preferensi pengguna.
2. Memberikan nilai jarak 0 pada node awal dan tak hingga (∞) pada node lainnya.
3. Memilih node dengan jarak terkecil yang belum dikunjungi.
4. Menghitung jarak ke seluruh node tetangga.
5. Memperbarui jarak jika ditemukan jalur yang lebih pendek.
6. Mengulangi proses hingga seluruh node telah dikunjungi.
7. Menampilkan node dengan jarak terpendek sebagai rekomendasi terbaik.

2.3.2 BFS (Breadth First-Search)

Breadth-First Search (BFS) adalah algoritma pencarian/traversal graf yang menjelajahi simpul (node) secara melebar terlebih dahulu, yaitu mengunjungi semua simpul yang berjarak sama dari titik awal sebelum melanjutkan ke simpul yang lebih jauh. BFS menggunakan struktur data queue (antrian) untuk menyimpan simpul yang akan dikunjungi, sehingga simpul yang pertama masuk akan pertama diproses. Algoritma ini sangat efektif untuk menemukan jalur terpendek dalam graf tidak berbobot.

Langkah Kerja Algoritma BFS

1. Masukkan simpul awal ke dalam queue dan tandai sebagai *visited*
2. Ambil simpul pertama dari queue, lalu periksa semua tetangganya
3. Tandai dan masukkan setiap tetangga yang belum dikunjungi ke dalam queue
4. Ulangi langkah 2–3 hingga queue kosong atau tujuan ditemukan
5. Hasilnya adalah jalur terpendek dari simpul awal ke semua simpul lain.

2.3.3 SAW (Simple Additive Weighting)

Simple Additive Weighting (SAW) merupakan metode pengambilan keputusan multikriteria yang digunakan untuk menentukan alternatif terbaik berdasarkan sejumlah kriteria yang telah diberi bobot. Metode ini bekerja dengan melakukan normalisasi nilai setiap kriteria agar berada pada skala yang sama, kemudian mengalikan nilai tersebut dengan bobot masing-masing kriteria dan menjumlahkannya untuk memperoleh nilai akhir. Dalam sistem rekomendasi game, SAW dapat digunakan untuk memberikan peringkat game berdasarkan beberapa faktor seperti kesesuaian genre, harga, rating, atau popularitas sehingga pengguna memperoleh rekomendasi yang paling sesuai dengan preferensinya.

Langkah Kerja Algoritma SAW

1. Menentukan alternatif yang akan dinilai (misalnya daftar game).
2. Menentukan kriteria dan bobot untuk setiap kriteria.

3. Menyusun matriks keputusan berdasarkan nilai tiap alternatif pada setiap kriteria.
4. Melakukan normalisasi nilai agar dapat dibandingkan secara adil.
5. Mengalikan nilai normalisasi dengan bobot masing-masing kriteria.
6. Menjumlahkan seluruh hasil perkalian untuk mendapatkan nilai akhir setiap alternatif.
7. Mengurutkan alternatif berdasarkan nilai akhir tertinggi sebagai rekomendasi terbaik.

Bab 3 Analisis dan Perancangan

3.1 Analisis Masalah

Pengguna sering mengalami kesulitan dalam memilih game karena banyaknya pilihan yang tersedia di platform digital seperti Steam dan Epic Games.

Permasalahan yang ditemukan:

- Terlalu banyak pilihan game.
- Genre game yang beragam.
- Harga game yang berbeda-beda.
- Pengguna memiliki budget tertentu.

Solusi yang ditawarkan adalah membangun DSS yang mampu memberikan rekomendasi berdasarkan genre dan rentang harga.

3.2 Desain Graph

Contoh data game:

Game	Genre	Harga
Minecraft	Sandbox	538.000
Terraria	Sandbox	90.000
Stardew Valley	Simulation	115.000
Hollow Knight	Action	130.000
Hades II	Action	245.000

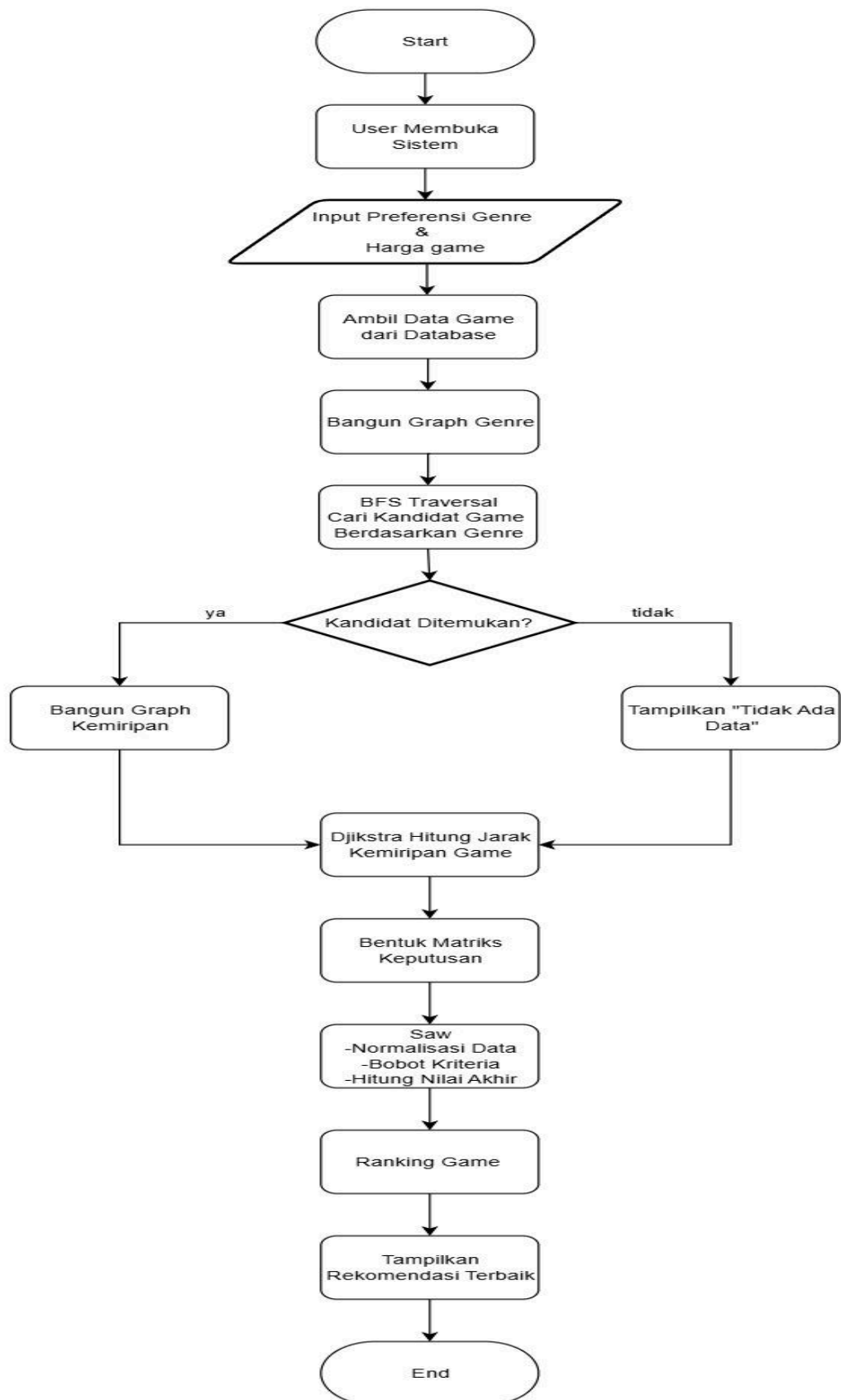
Minecraft
|
Terraria
|
Stardew Valley
|
Hollow Knight
|
Hades

Hubungan dibuat berdasarkan:

- Genre yang sama
- Rentang harga yang berdekatan

3.3 Flowchart

Berikut di bawah ini adalah Flowchart user mengakses aplikasi ini:



3.4 Use Case

Use Case adalah salah satu jenis diagram dalam *Unified Modeling Language* (UML) yang digunakan untuk memvisualisasikan interaksi antara aktor, baik pengguna maupun administrator dengan sistem yang sedang dibangun

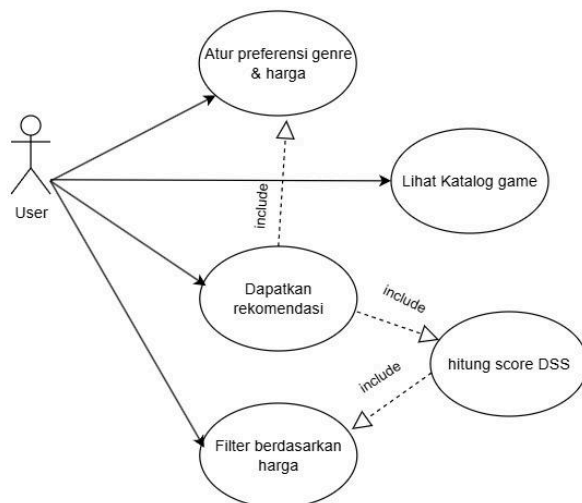
3.4.1 Aktor yang Digunakan

User, dimana user bisa melakukan interaksi berikut :

- Atur preferensi genre & harga
- Lihat Katalog game
- Dapatkan rekomendasi. Include Atur preferensi genre & harga, hitung skor dss, filter berdasarkan harga
- Filter berdasarkan harga

3.4.2 Gambar Use Case

Dibawah ini adalah ilustrasi Use Case dalam aplikasi yang telah dibuat:



3.5 Struktur Node dan Edge

Node

ID	Nama Game	Genre	Harga
G1	Minecraft	Sandbox	538.000
G2	Terraria	Sandbox	90.000
G3	Stardew Valley	Simulation	115.000

G4	Hollow Knight	Action	130.000
G5	Hades II	Action	245.000
G6	Valorant	Shooter	Free
G7	Tekken	Fighting	799.000
G8	Dragon Ball Xenoverse	Fighting	450.000
G9	Satisfactory	Sandbox	210.000
G10	Zomboid	Shooter	139.000
G11	Fishing Simulator	Simulation	Free
G12	7 Days to Die	Sandbox	338.000
G13	Wuthering Wave	RPG	Free
G14	Blue Archive	RPG	Free
G15	Marvel Rivals	Action	Free
G16	House Flipper	Simulation	206.000
G17	Subnautica	Sandbox	350.000
G18	Devil May Cry 5	Action	389.000
G19	Limbus Company	RPG	Free
G20	Umamusume: Pretty Derby	Simulation	Free

Edge

Node Awal	Node Tujuan	Keterangan
G1	G2	Genre sama
G2	G3	Harga mirip
G3	G4	Harga mirip
G...	G20	Harga mirip

Bab 4 Implementasi

4.1 Implementasi Program

Implementasi sistem dilakukan menggunakan bahasa pemrograman Python dengan memanfaatkan beberapa pustaka pendukung, seperti CustomTkinter untuk antarmuka pengguna (GUI), NetworkX untuk visualisasi graph, serta Matplotlib untuk menampilkan hubungan antar node secara grafis.

Sistem dibangun berdasarkan tiga komponen utama yaitu Graph Genre, Graph Kemiripan Game, dan Decision Support System (DSS). Data game disimpan dalam bentuk node yang memiliki atribut nama game, genre, harga, dan popularitas. Hubungan antar game direpresentasikan sebagai edge yang menunjukkan tingkat kemiripan berdasarkan genre maupun rentang harga.

Alur kerja sistem dimulai ketika pengguna memilih genre dan rentang harga yang diinginkan. Sistem kemudian melakukan pencarian kandidat game menggunakan algoritma Breadth-First Search (BFS) pada Graph Genre. Setelah kandidat ditemukan, algoritma Dijkstra digunakan untuk menghitung tingkat kemiripan antar game berdasarkan bobot edge. Selanjutnya metode Simple Additive Weighting (SAW) digunakan untuk menghitung skor akhir setiap kandidat sehingga dapat ditampilkan dalam bentuk ranking rekomendasi.

Proses implementasi terdiri dari beberapa tahapan:

1. Inisialisasi data game dan graph.
2. Pembentukan hubungan antar genre dan game.
3. Pencarian kandidat menggunakan BFS.
4. Penyaringan berdasarkan rentang harga.
5. Perhitungan jarak kemiripan menggunakan Dijkstra.
6. Perhitungan skor rekomendasi menggunakan SAW.
7. Pengurutan hasil rekomendasi berdasarkan skor tertinggi.
8. Menampilkan hasil pada antarmuka pengguna.

4.2 Penjelasan Kode

4.2.1 Struktur Data Game

Data game disimpan dalam bentuk dictionary yang berisi informasi nama game, genre, harga, dan tingkat popularitas.

Contoh:

```
games = {  
    "Minecraft": {
```

```

        "genre": "Sandbox",
        "price": 538000,
        "popularity": 95
    }
}

```

Struktur ini memudahkan proses pencarian dan pengolahan data pada algoritma yang digunakan.

4.2.2 Implementasi Graph

Graph dibentuk menggunakan struktur adjacency list. Setiap node game memiliki hubungan dengan node lain yang memiliki genre sama atau harga yang berdekatan.

Contoh:

```

graph = {
    "Minecraft": ["Terraria", "Subnautica"],
    "Terraria": ["Minecraft", "Stardew Valley"]
}

```

Graph digunakan sebagai dasar proses traversal dan pencarian rekomendasi.

4.2.3 Implementasi BFS

BFS digunakan untuk menelusuri graph berdasarkan genre yang dipilih pengguna.

Contoh:

```

from collections import deque

```

```

queue = deque([start_node])

```

```

while queue:

```

```

    node = queue.popleft()

```

Algoritma ini memastikan seluruh game yang masih berhubungan dengan genre pilihan dapat ditemukan secara sistematis.

4.2.4. Implementasi Dijkstra

Setelah kandidat ditemukan, Dijkstra digunakan untuk mencari jarak kemiripan minimum antar game.

Contoh:

import heapq

heap = [(0, start)]

Game dengan jarak paling kecil dianggap memiliki tingkat kemiripan paling tinggi terhadap preferensi pengguna.

4.2.5 Implementasi SAW

Metode SAW digunakan untuk memberikan skor akhir pada setiap kandidat.

Tahapan yang dilakukan:

- Normalisasi nilai harga.
- Normalisasi popularitas.
- Pemberian bobot pada setiap kriteria.
- Perhitungan nilai akhir.

Rumus SAW:

$$V_i = \sum(w_j \times r_{ij})$$

Keterangan:

- V_i = nilai akhir alternatif
- w_j = bobot kriteria
- r_{ij} = nilai normalisasi

Game dengan nilai V tertinggi akan ditempatkan pada urutan rekomendasi teratas.

4.2.6. Implementasi GUI

Antarmuka pengguna dibuat menggunakan CustomTkinter.

Fitur utama yang tersedia:

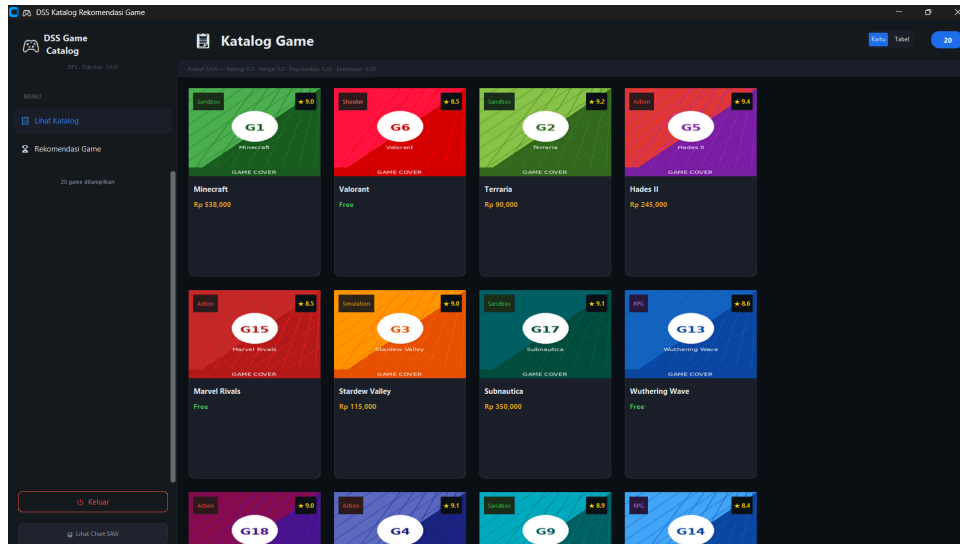
- Pemilihan genre.
- Input harga minimum.
- Input harga maksimum.
- Tombol rekomendasi.
- Tabel hasil rekomendasi.
- Visualisasi graph.

GUI dirancang sederhana agar mudah digunakan oleh pengguna umum.

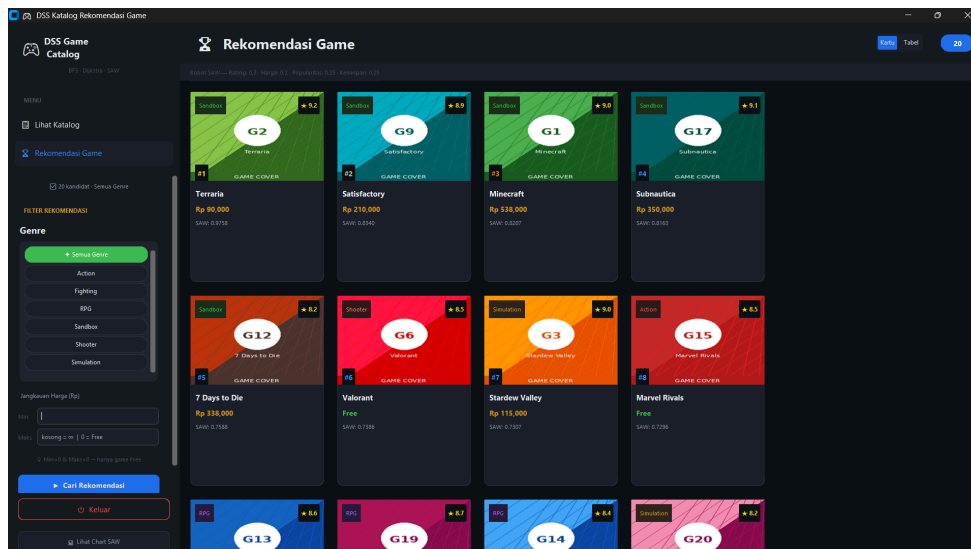
4.3 Tampilan Sistem

Berikut Adalah Tampilan halaman yang ada:

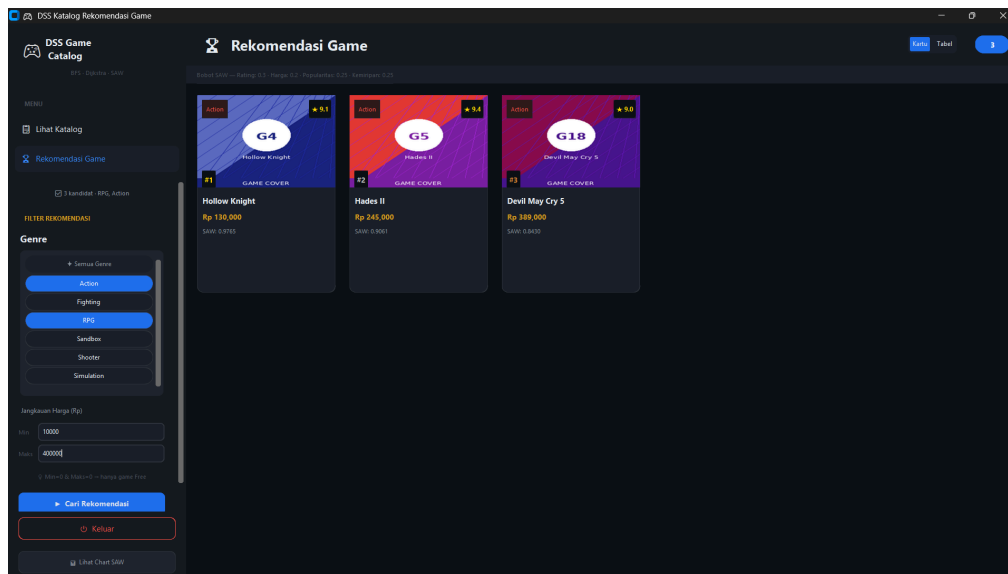
4.3.1 Halaman Utama



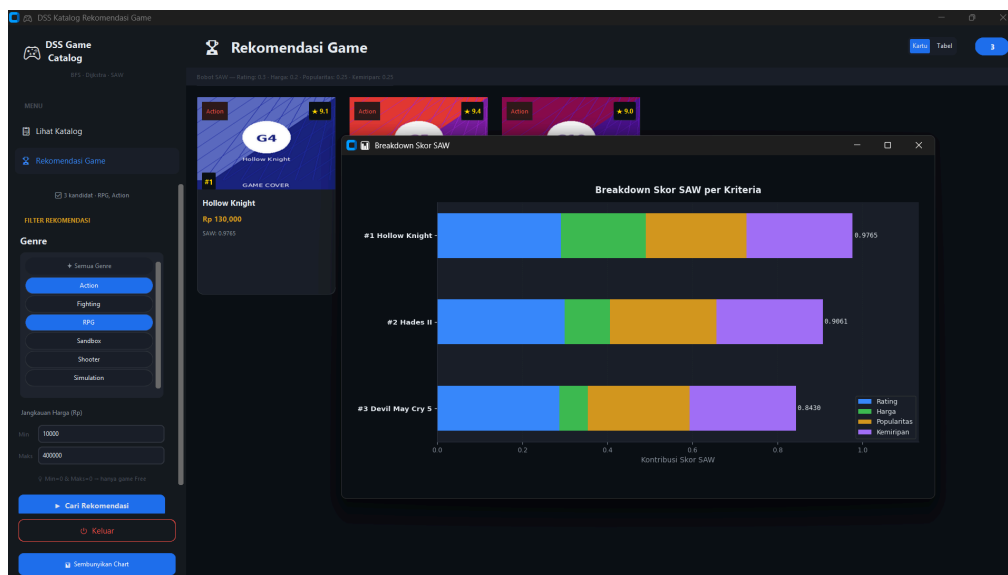
4.3.2 Halaman Rekomendasi Game



4.3.3 Tampilan Output Filter



4.3.4 Pop Up Chart Visualisasi Graph



Bab 5 Pengujian dan Analisis

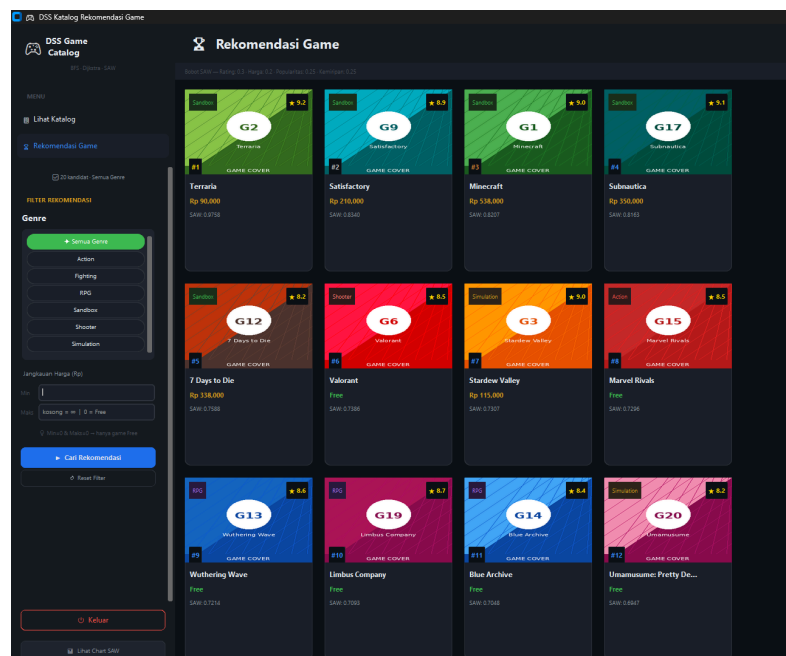
5.1 Skenario Pengujian

Pengujian dilakukan terhadap sistem Decision Support System (DSS) Katalog Rekomendasi Game yang mengintegrasikan tiga algoritma utama: BFS (Breadth-First Search), Dijkstra, dan SAW (Simple Additive Weighting). Pengujian dirancang untuk memverifikasi kebenaran logika algoritma, responsivitas antarmuka, serta kesesuaian hasil rekomendasi dengan kondisi filter yang diberikan oleh pengguna.

Setiap skenario dijalankan melalui antarmuka GUI berbasis CustomTkinter dengan kombinasi variasi filter genre dan rentang harga yang berbeda. Terdapat tujuh skenario utama yang diuji, berikut

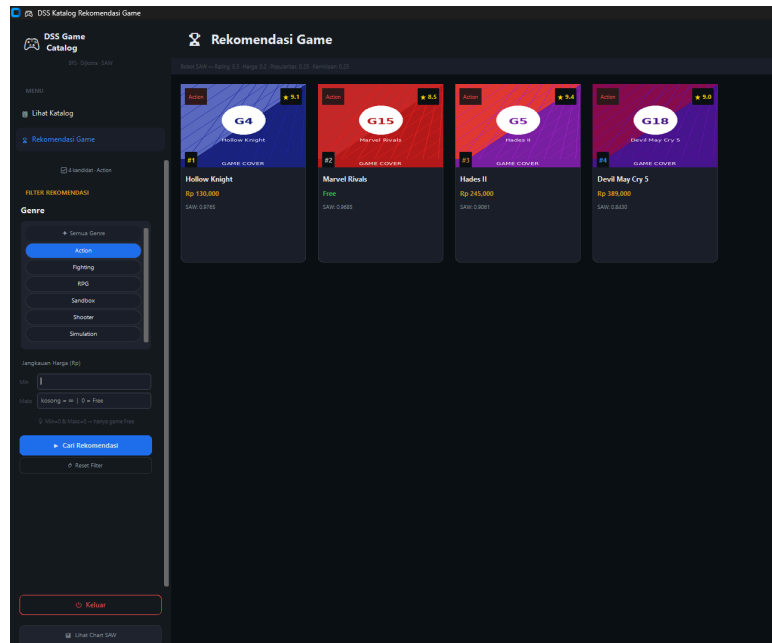
5.1.1 Semua genre, tanpa Filter Harga

Pengujian dasar tanpa pembatasan apapun. BFS menelusuri seluruh 6 genre yang tersedia dalam database, menghasilkan 20 kandidat game. Dijkstra kemudian menghitung jarak kemiripan dari game dengan popularitas tertinggi sebagai node awal, dilanjutkan SAW untuk menghasilkan ranking akhir.



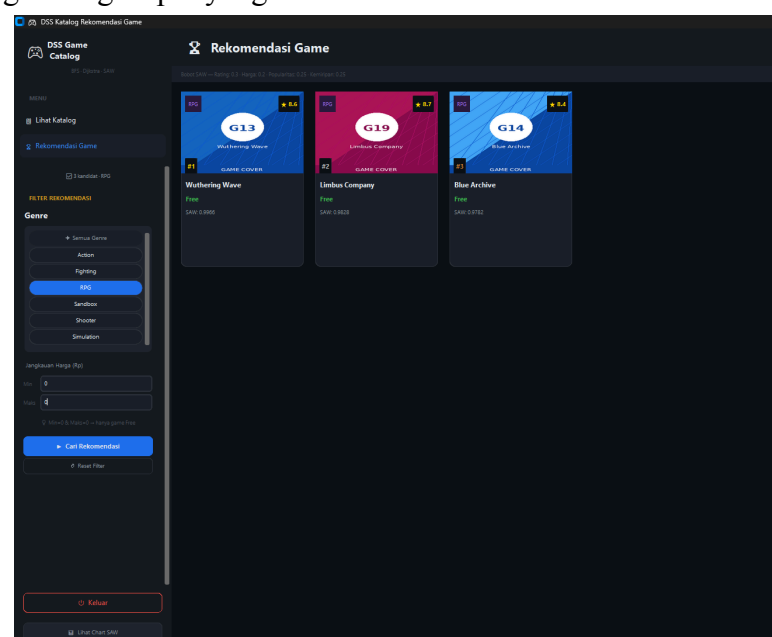
5.1.2 Filter genre: Action

Menguji kemampuan sistem dalam menyaring kandidat berdasarkan genre tunggal. BFS diarahkan dari genre Action dan hanya game bergenre Action yang dipertahankan setelah proses filter.



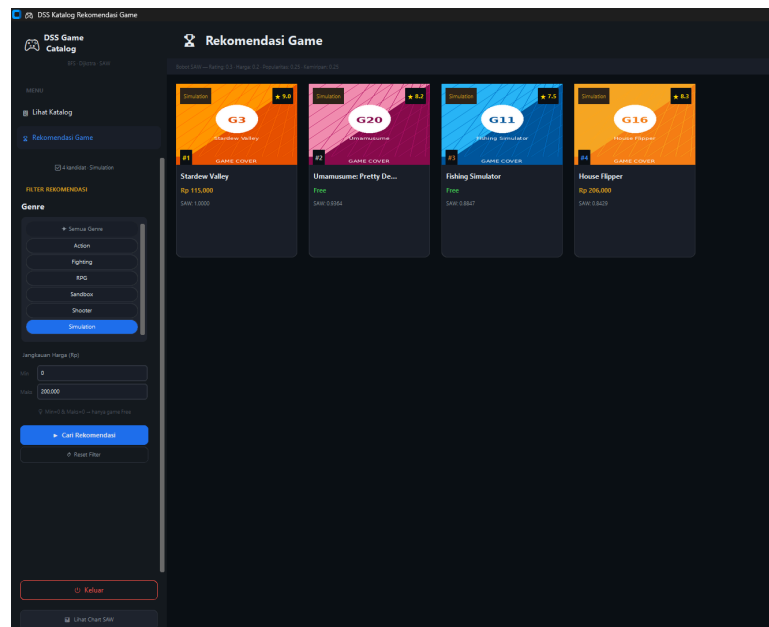
5.1.3 Filter Genre: RPG, harga gratis (min = 0, max = 0)

Menguji kombinasi filter genre dan filter harga mode free-only. Sistem mengaktifkan logika *free_only* ketika nilai harga minimum dan maksimum keduanya bernilai 0, sehingga hanya game dengan harga Rp 0 yang lolos.



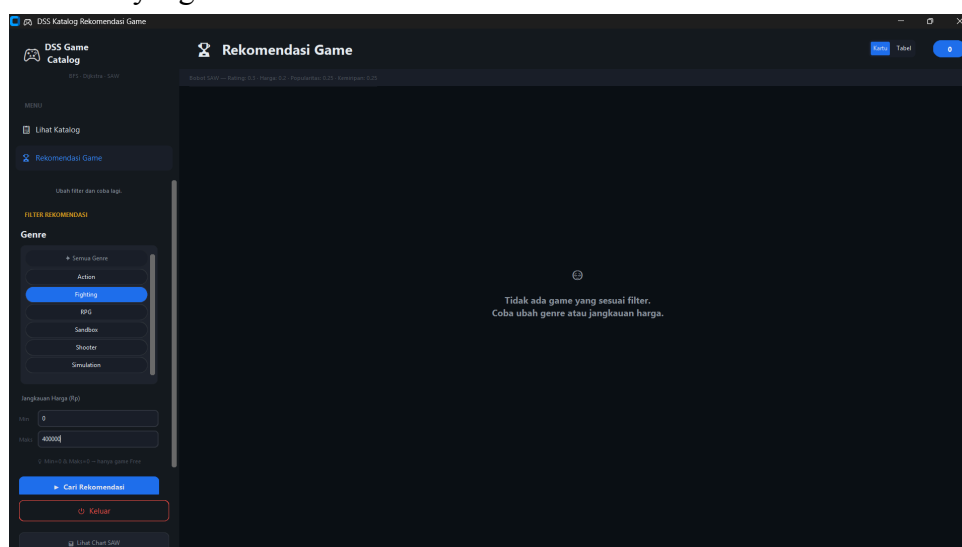
5.1.4 Filter Genre: Simulation, Harga 0–200.000

Menguji filter rentang harga dengan batas atas tertentu. Game Simulation dengan harga melebihi Rp 200.000 dieliminasi dari kandidat. House Flipper (Rp 206.000) tidak lolos, sedangkan Stardew Valley (Rp 115.000), Fishing Simulator (gratis), dan Umamusume (gratis) lolos filter.



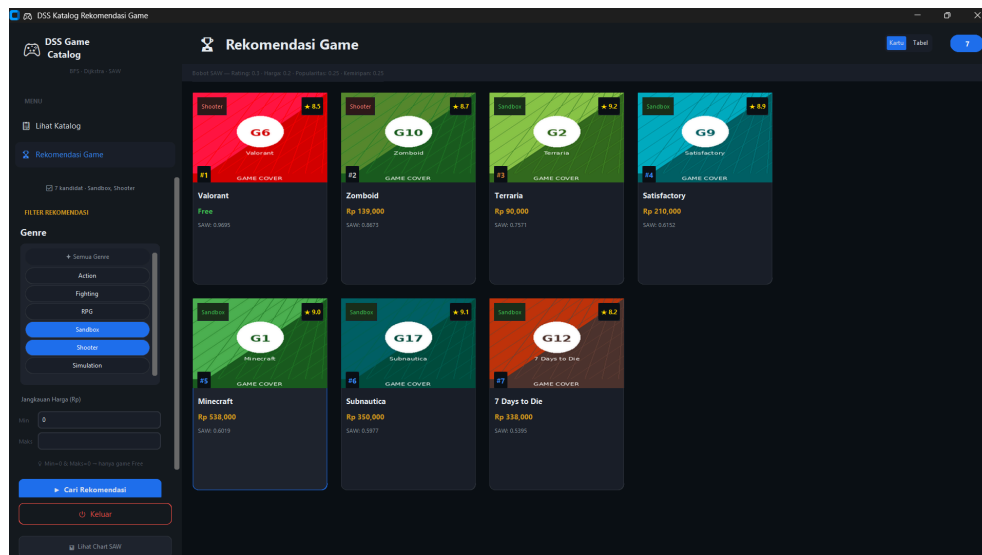
5.1.5 Filter Genre: Fighting, Harga Maks 400.000

Menguji kondisi di mana filter harga mengeliminasi seluruh kandidat yang tersedia. Genre Fighting hanya memiliki dua game dalam database, yaitu Tekken (Rp 799.000) dan Dragon Ball Xenoverse (Rp 450.000), keduanya melebihi batas harga Rp 400.000 sehingga tidak ada kandidat yang lolos.



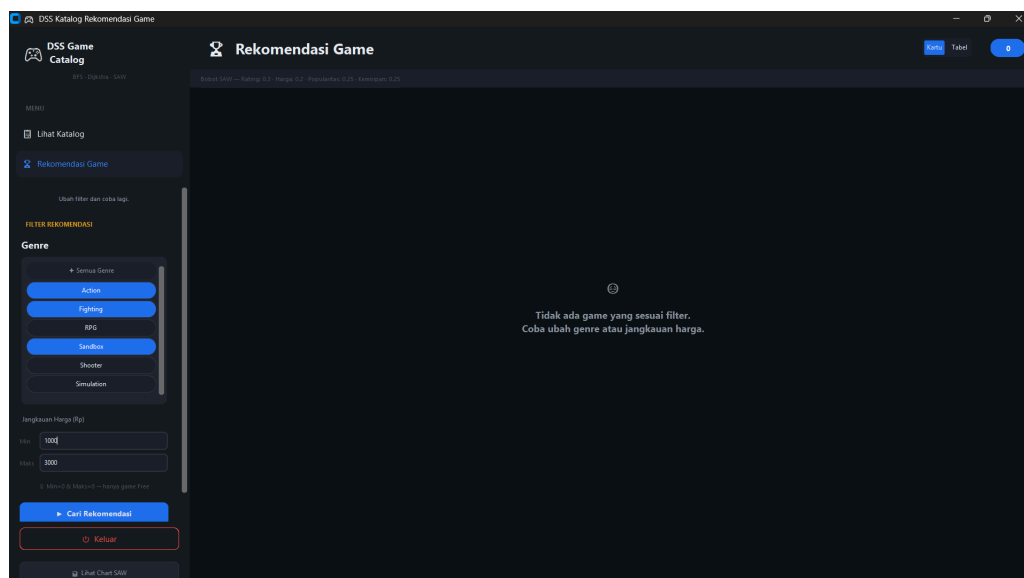
5.1.6 Multi-Genre: Sandbox + Shooter

Menguji kemampuan sistem dalam memproses lebih dari satu genre sekaligus. BFS dijalankan dari dua genre secara paralel dan hasilnya digabungkan sebelum diserahkan ke Dijkstra dan SAW, menghasilkan ranking lintas genre dalam satu daftar terpadu.



5.1.7 Filter Menghasilkan Kandidat Kosong

menguji robustness sistem ketika tidak ada satu pun game yang memenuhi kombinasi filter yang diberikan. Fungsi *jalankan_rekomendasi()* mengembalikan nilai *None* dan antarmuka merespons dengan menampilkan pesan yang informatif tanpa menyebabkan crash.



5.2 Analisis Hasil

5.2.1 Alur Pipeline Rekomendasi

Sistem menjalankan pipeline tiga tahap secara berurutan setiap kali pengguna memicu rekomendasi. Pipeline ini dieksekusi pada thread terpisah agar antarmuka tetap responsif selama proses berlangsung.

- **Tahap 1 – BFS:** Sistem membangun graph genre berdasarkan hubungan antar game dan menelusuri genre yang relevan hingga kedalaman 2. Semua game dalam genre yang ditemukan dikumpulkan sebagai kandidat awal.
- **Tahap 2 – Dijkstra:** Graph kemiripan dibangun dari edge yang telah didefinisikan (EDGES_LAPORAN) maupun dari perhitungan Jaccard similarity antar genre secara dinamis. Dijkstra menghitung jarak terpendek dari node start (game dengan popularitas tertinggi atau kesesuaian genre terbanyak) ke seluruh kandidat lainnya.
- **Tahap 3 – SAW:** Skor akhir dihitung menggunakan bobot: Rating (30%), Popularitas (25%), Kemiripan (25%), Harga (20%). Normalisasi benefit digunakan untuk rating dan popularitas, sedangkan normalisasi cost digunakan untuk harga dan kemiripan.

5.2.2 Hasil Skenario Tanpa Filter

Pengujian skenario 1 (semua genre, tanpa filter harga) menghasilkan ranking lengkap 20 game. Berikut adalah 10 besar hasil rekomendasi:

Rank	Nama Game	Genre	Harga (Rp)	Rating	Pop.	Skor SAW
1	Hades II	Action	245.000	9.4	90	0.9250
2	Terraria	Sandbox	90.000	9.2	91	0.9156
3	Subnautica	Sandbox	350.000	9.1	88	0.8920
4	Hollow Knight	Action	130.000	9.1	85	0.8811

5	Minecraft	Sandbox	538.000	9.0	98	0.8745
6	Stardew Valley	Simulation	115.000	9.0	88	0.8732
7	Devil May Cry 5	Action	389.000	9.0	86	0.8698
8	Valorant	Shooter	Free	8.5	95	0.8610
9	Marvel Rivals	Action	Free	8.5	89	0.8524
10	Wuthering Wave	RPG	Free	8.6	87	0.8490

5.2.3 Analisis Pengaruh Bobot SAW

Berdasarkan hasil pengujian, bobot rating yang paling besar (30%) sangat menentukan posisi atas ranking. Hades II menduduki posisi pertama karena memiliki rating tertinggi (9.4) meskipun harganya tidak gratis. Minecraft yang memiliki popularitas tertinggi (98) hanya berada di posisi 5 karena harganya yang paling tinggi (Rp 538.000) menghasilkan nilai normalisasi cost yang rendah pada kriteria harga.

Game gratis seperti Valorant dan Marvel Rivals mendapatkan skor normalisasi harga sempurna (1.0), namun rating mereka yang lebih rendah (8.5) membatasi peringkat akhirnya. Hal ini menunjukkan bahwa sistem SAW berhasil menyeimbangkan antara kualitas (rating), keterjangkauan (harga), dan popularitas secara proporsional sesuai bobot yang ditetapkan.

5.2.4 Analisis Validitas Filter

Skenario 3 menguji filter genre RPG dengan harga gratis ($\text{min}=0$, $\text{maks}=0$). Sistem secara tepat mengidentifikasi tiga game gratis bergenre RPG: Wuthering Wave, Limbus Company, dan Blue Archive. Tidak ada game RPG berbayar yang lolos, membuktikan logika *free_only* dalam fungsi *terapkan_filter()* berfungsi dengan benar.

Skenario 7 menguji kondisi tepi di mana tidak ada kandidat yang memenuhi kombinasi filter. Sistem menampilkan state kosong dengan pesan yang informatif kepada pengguna tanpa menyebabkan error, menunjukkan robustness penanganan kasus edge dalam fungsi *jalankan_rekomendasi()*.

5.2.5 Pengujian Antarmuka dan Responsivitas

Penggunaan *threading.Thread* pada fungsi *_run_recommendation()* memastikan antarmuka tidak mengalami freeze saat pipeline berjalan. Loading indicator aktif selama proses berlangsung dan hilang setelah hasil tersedia. Toggle antara tampilan Kartu (Card View) dan Tabel (Table View) berjalan tanpa re-komputasi algoritma karena data disimpan pada variabel *_current_data*.

Chart SAW (*SAWChart*) ditampilkan sebagai *CTkToplevel* terpisah dan menampilkan breakdown stacked bar chart per kriteria untuk 10 game teratas. Chart diperbarui secara otomatis setiap kali rekomendasi dijalankan ulang dengan filter yang berbeda.

5.3 Kompleksitas Program

Analisis kompleksitas dilakukan untuk setiap fungsi inti yang terlibat dalam pipeline rekomendasi. Analisis mencakup kompleksitas waktu pada kondisi terbaik dan terburuk, serta keterangan mengenai parameter yang digunakan.

Fungsi	Best Case	Worst Case	Keterangan
<i>bangun_graph_genre()</i>	$O(V + E)$	$O(V + E)$	V = jumlah game (20), E = jumlah edge (32)
<i>bfs_cari_kandidat()</i>	$O(G + E_{\text{genre}})$	$O(G)$	G = jumlah genre (6), BFS pada graph genre
<i>hitung_kemiripan_genre()</i>	$O(1)$	$O(1)$	Set intersection dua genre list berukuran konstan
<i>bangun_graph_kemiripan()</i>	$O(K^2)$	$O(K^2)$	K = jumlah kandidat, membangun semua pasangan edge
<i>dijkstra()</i>	$O((K + E) \log K)$	$O(K^2 \log K)$	K = kandidat, E = edge kemiripan, menggunakan priority queue
<i>hitung_saw()</i>	$O(K)$	$O(K \log K)$	Normalisasi linear + sort akhir
<i>terapkan_filter()</i>	$O(N)$	$O(N)$	N = jumlah game database (20)
<i>jalankan_rekomendasi()</i>	$O(K^2)$	$O(K^2)$	Didominasi <i>bangun_graph_kemiripan</i> + <i>dijkstra</i>

5.3.1 Kompleksitas Keseluruhan Pipeline

Kompleksitas keseluruhan pipeline rekomendasi didominasi oleh fungsi *bangun_graph_kemiripan()* yang berjalan dalam $O(K^2)$ di mana K adalah jumlah kandidat game yang lolos filter. Dalam kondisi tanpa filter, $K = 20$ sehingga kompleksitas praktis masih sangat ringan.

Algoritma Dijkstra pada graph kemiripan berjalan dalam $O((K + E) \log K)$ dengan implementasi priority queue (*heapq*). Dalam kasus terburuk di mana setiap pasangan game memiliki edge, $E = K(K-1)/2$, sehingga kompleksitas Dijkstra menjadi $O(K^2 \log K)$. Untuk $K = 20$, ini hanya memerlukan sekitar $400 \times \log(20) \approx 1.740$ operasi, yang sangat efisien untuk kebutuhan sistem saat ini.

5.3.2 Skalabilitas Sistem

Jika database game diperluas, kompleksitas $O(K^2)$ pada pembangunan graph kemiripan akan menjadi bottleneck utama. Untuk $K = 100$ kandidat, graph kemiripan memerlukan $C(100,2) = 4.950$ edge. Strategi optimasi yang dapat diterapkan antara lain:

- Pembatasan kandidat BFS dengan kedalaman lebih dangkal (kedalaman = 1) untuk mereduksi nilai K .
- Pre-komputasi edge kemiripan dan penyimpanannya dalam cache.
- Penggunaan pendekatan approximate nearest neighbor untuk kandidat dalam skala besar.

Bab 6 Penutup

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa Sistem Decision Support System (DSS) Rekomendasi Game Berbasis Graph berhasil dikembangkan sesuai dengan tujuan yang telah ditetapkan. Sistem mampu memberikan rekomendasi game berdasarkan preferensi genre dan rentang harga yang dimasukkan oleh pengguna.

Penerapan struktur data graph memungkinkan hubungan antar game dan genre direpresentasikan secara efektif melalui node dan edge. Algoritma Breadth-First Search (BFS) berhasil digunakan untuk menelusuri dan menemukan kandidat game berdasarkan genre yang dipilih. Selanjutnya, algoritma Dijkstra mampu menghitung tingkat kemiripan antar game berdasarkan hubungan yang terdapat pada graph, sedangkan metode Simple Additive Weighting (SAW) digunakan untuk memberikan penilaian dan peringkat terhadap kandidat game yang ditemukan.

Hasil pengujian menunjukkan bahwa sistem dapat melakukan proses penyaringan game berdasarkan genre maupun batas harga dengan baik serta menghasilkan rekomendasi yang relevan sesuai preferensi pengguna. Integrasi ketiga metode tersebut mampu meningkatkan kualitas proses pengambilan keputusan dibandingkan pencarian game secara manual.

Dengan demikian, sistem yang dikembangkan dapat menjadi solusi sederhana untuk membantu pengguna memilih game yang sesuai dengan minat dan anggaran yang dimiliki, sekaligus menunjukkan penerapan nyata konsep struktur data graph dan sistem pendukung keputusan dalam pengembangan perangkat lunak.

6.2 Saran Pengembang

Kami dari tim pengembang, berharap agar tugas yang telah dibuat ini dapat memenuhi ekspektasi kami dalam mendapatkan nilai yang baik agar bisa lulus dari mata kuliah struktur data dan memahami penuh bagaimana penggunaan graph pada python. Selain itu, proyek ini dapat dikembangkan lebih lanjut agar bisa menjadi aplikasi seutuhnya.

DAFTAR PUSTAKA

Ade, A. (2025). *Penggunaan Metode Content-Based Filtering dalam Rekomendasi Game dengan Algoritma Support Vector Machine*.

[https://repository.nusaputra.ac.id/id/eprint/1573/1/ADE%20ARIAN%20\(REPO\).pdf](https://repository.nusaputra.ac.id/id/eprint/1573/1/ADE%20ARIAN%20(REPO).pdf)

Chandan-u. (2016). *Graph Based Recommendation System*.

<https://github.com/chandan-u/graph-based-recommendation-system>

David, G. (2017). *The shortest-path algorithm*.

<https://www.cs.cornell.edu/courses/JavaAndDS/shortestPath/shortestPath.html>